

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Bayesian Scaffold Monitor</title>
<style>
  @import url('https://fonts.googleapis.com/css2?
family=Share+Tech+Mono&family=Bebas+Neue&display=swap');
```

```
:root {
  --bg: #080c0a;
  --panel: #0c110e;
  --border: #1a2e20;
  --human: #e85d4a;
  --ai: #4ae8b0;
  --route0: #4ae8b0;
  --route1: #a0e870;
  --route2: #f0c060;
  --route3: #e8954a;
  --gold: #f0c060;
  --dim: #4a6050;
  --text: #b8d8c0;
  --warn: #e8954a;
  --scanline: rgba(0,255,80,0.02);
}
```

```
* { margin:0; padding:0; box-sizing:border-box; }
```

```
body {
  background: var(--bg);
  color: var(--text);
  font-family: 'Share Tech Mono', monospace;
  min-height: 100vh;
  padding: 20px;
  display: flex;
  flex-direction: column;
  align-items: center;
  background-image:
    repeating-linear-gradient(
```

```
    0deg,  
    transparent,  
    transparent 2px,  
    var(--scanline) 2px,  
    var(--scanline) 4px  
  );  
}
```

```
header {  
  text-align: center;  
  margin-bottom: 24px;  
  width: 100%;  
  max-width: 960px;  
}
```

```
h1 {  
  font-family: 'Bebas Neue', sans-serif;  
  font-size: clamp(1.8rem, 4vw, 3rem);  
  letter-spacing: 0.18em;  
  color: var(--ai);  
  text-shadow: 0 0 20px rgba(74,232,176,0.3);  
}
```

```
.sub {  
  font-size: 0.68rem;  
  color: var(--dim);  
  letter-spacing: 0.25em;  
  margin-top: 2px;  
}
```

```
.grid {  
  width: 100%;  
  max-width: 960px;  
  display: grid;  
  grid-template-columns: 1fr 1fr;  
  gap: 12px;  
}
```

```
.panel {  
  background: var(--panel);  
  border: 1px solid var(--border);  
}
```

```
padding: 16px;
position: relative;
}
```

```
.panel.full { grid-column: 1 / -1; }
```

```
.plabel {
  font-family: 'Bebas Neue', sans-serif;
  font-size: 0.78rem;
  letter-spacing: 0.22em;
  color: var(--dim);
  border-bottom: 1px solid var(--border);
  padding-bottom: 8px;
  margin-bottom: 14px;
}
```

```
/* —— Upload —— */
```

```
.upload-zone {
  border: 1px dashed var(--border);
  padding: 20px;
  text-align: center;
  cursor: pointer;
  transition: border-color 0.2s, background 0.2s;
  font-size: 0.8rem;
  color: var(--dim);
}
.upload-zone:hover, .upload-zone.drag {
  border-color: var(--ai);
  background: rgba(74,232,176,0.04);
  color: var(--ai);
}
.upload-zone input { display:none; }
```

```
/* —— Belief bars —— */
```

```
.duel { display:flex; align-items:center; gap:10px; margin-bottom:10px; }
.alabel { font-family:'Bebas Neue',sans-serif; font-size:1rem; letter-spacing:0.1em; width:70px; }
.alabel.h { color:var(--human); text-align:right; }
.alabel.a { color:var(--ai); }
```

```
.btrack {
  flex:1; height:24px; background:#060a07;
```

```

border:1px solid var(--border); overflow:hidden; position:relative;
}
.bfill {
height:100%; transition: width 0.8s cubic-bezier(0.23,1,0.32,1);
display:flex; align-items:center;
}
.bfill.h { background: linear-gradient(90deg,#4a1008,var(--human)); float:right; justify-
content:flex-start; }
.bfill.a { background: linear-gradient(90deg,var(--ai),#0a4030); justify-content:flex-end; }
.bpct { font-size:0.72rem; font-weight:700; padding:0 6px; color:#000; }
.bfill.h .bpct { color:#fff; }

/* —— Stats strip —— */
.stats { display:flex; gap:10px; flex-wrap:wrap; }
.stat { flex:1; min-width:80px; background:#060a07; border:1px solid var(--border);
padding:10px; text-align:center; }
.sval { font-family:'Bebas Neue',sans-serif; font-size:1.6rem; color:var(--gold); letter-
spacing:0.05em; line-height:1; }
.slbl { font-size:0.58rem; color:var(--dim); letter-spacing:0.18em; text-transform:uppercase;
margin-top:3px; }

/* —— Bias radar —— */
.bias-list { display:flex; flex-direction:column; gap:7px; }
.bias-row { display:flex; align-items:center; gap:8px; }
.bname { font-size:0.68rem; width:150px; color:var(--text); flex-shrink:0; text-
transform:capitalize; }
.bbar-track { flex:1; height:12px; background:#060a07; border:1px solid var(--border);
overflow:hidden; }
.bbar-fill { height:100%; background: linear-gradient(90deg,#1a4030,var(--ai)); transition:width
0.7s ease; }
.bcount { font-size:0.68rem; width:28px; text-align:right; color:var(--gold); }

/* —— Outcome distribution —— */
.outcome-list { display:flex; flex-direction:column; gap:7px; }
.ow { display:flex; align-items:center; gap:8px; }
.owname { font-size:0.68rem; width:160px; color:var(--text); flex-shrink:0; }
.otrack { flex:1; height:12px; background:#060a07; border:1px solid var(--border);
overflow:hidden; }
.ofill { height:100%; transition:width 0.7s ease; }
.opct { font-size:0.68rem; width:34px; text-align:right; color:var(--gold); }

```

```
/* — Assertiveness dial — */
```

```
.dial-wrap { display:flex; flex-direction:column; align-items:center; gap:8px; }  
canvas#dial { display:block; }  
.dial-label { font-size:0.7rem; color:var(--dim); letter-spacing:0.15em; text-align:center; }
```

```
/* — Recent log — */
```

```
.rlog { height:130px; overflow-y:auto; font-size:0.7rem; line-height:1.8; color:var(--dim); }  
.rlog::-webkit-scrollbar { width:2px; }  
.rlog::-webkit-scrollbar-thumb { background:var(--border); }  
.rlog-entry { border-bottom:1px solid #0c110e; padding:1px 0; }  
.ev-right { color:var(--ai); }  
.ev-clarify{ color:var(--gold); }  
.ev-wrong { color:var(--human); }
```

```
/* — Status bar — */
```

```
.statusbar {  
  grid-column:1/-1;  
  background:#060a07;  
  border:1px solid var(--border);  
  padding:8px 14px;  
  font-size:0.68rem;  
  color:var(--dim);  
  display:flex;  
  justify-content:space-between;  
  flex-wrap:wrap;  
  gap:6px;  
}  
.statusbar span { color:var(--ai); }
```

```
/* — Manual sim controls — */
```

```
.controls { display:flex; gap:8px; flex-wrap:wrap; margin-top:10px; }  
.btn {  
  font-family:'Bebas Neue',sans-serif;  
  letter-spacing:0.14em;  
  font-size:0.85rem;  
  padding:8px 18px;  
  border:1px solid;  
  background:transparent;  
  cursor:pointer;  
  transition:all 0.2s;  
  flex:1;
```

```

}
.btn-ai { border-color:var(--ai); color:var(--ai); }
.btn-h { border-color:var(--human); color:var(--human); }
.btn-reset { border-color:var(--dim); color:var(--dim); }
.btn-load { border-color:var(--gold); color:var(--gold); }
.btn-ai:hover { background:rgba(74,232,176,0.08); }
.btn-h:hover { background:rgba(232,93,74,0.08); }
.btn-reset:hover { background:rgba(100,100,100,0.08); }
.btn-load:hover { background:rgba(240,192,96,0.08); }

.select-bias {
  font-family:'Share Tech Mono',monospace;
  font-size:0.75rem;
  background:#060a07;
  border:1px solid var(--border);
  color:var(--text);
  padding:8px 10px;
  flex:2;
  cursor:pointer;
}

/* —— Route badge —— */
.route-badge {
  display:inline-block;
  font-family:'Bebas Neue',sans-serif;
  letter-spacing:0.12em;
  font-size:0.9rem;
  padding:4px 12px;
  border:1px solid;
  margin-bottom:8px;
}

.rb-comply { border-color:var(--route0); color:var(--route0); }
.rb-reframe { border-color:var(--route1); color:var(--route1); }
.rb-clarify { border-color:var(--route2); color:var(--route2); }
.rb-challenge { border-color:var(--route3); color:var(--route3); }

.verdict-text { font-size:0.78rem; line-height:1.65; font-style:italic; color:var(--text); margin-top:6px; }

@keyframes flick { 0%{opacity:0.1} 60%{opacity:1} 100%{opacity:1} }
.flick { animation: flick 0.35s ease; }

```

```

</style>
</head>
<body>

<header>
  <h1>Bayesian Scaffold Monitor</h1>
  <p class="sub">Live session memory · AI ↔ Human belief posteriors · Route decisions</p>
</header>

<div class="grid">

  <!-- FILE LOAD -->
  <div class="panel full">
    <div class="plabel">LOAD MEMORY SNAPSHOT — scaffold_memory.json</div>
    <div class="upload-zone" id="dropZone">
      <input type="file" id="fileInput" accept=".json">
      <div>Drop <code>scaffold_memory.json</code> here, or click to browse<br>
      <small style="color:#333;margin-top:6px;display:block">No file? Use the manual simulator
below.</small></div>
    </div>
  </div>

  <!-- BELIEF BARS -->
  <div class="panel full" id="beliefPanel">
    <div class="plabel">POSTERIOR BELIEFS — CURRENT STATE</div>
    <div class="duel">
      <span class="alabel h">HUMAN</span>
      <div class="btrack" style="direction:rtl;">
        <div class="bfill h" id="hBar" style="width:50%">
          <span class="bpct" id="hPct">50%</span>
        </div>
      </div>
    </div>
    <div class="duel">
      <span class="alabel a">AI</span>
      <div class="btrack">
        <div class="bfill a" id="aBar" style="width:50%">
          <span class="bpct" id="aPct">50%</span>
        </div>
      </div>
    </div>
  </div>

```

</div>

<!-- STATS -->

<div class="panel full">

<div class="plabel">SESSION STATISTICS</div>

<div class="stats">

<div class="stat"><div class="sval" id="stRounds">0</div><div class="slbl">Rounds</div></div>

<div class="stat"><div class="sval" id="stSessions">0</div><div class="slbl">Sessions</div></div>

<div class="stat"><div class="sval" id="stHits">—</div><div class="slbl">Hit Rate</div></div>

<div class="stat"><div class="sval" id="stClarify">—</div><div class="slbl">Clarify Rate</div></div>

<div class="stat"><div class="sval" id="stWrong">—</div><div class="slbl">Wrong Rate</div></div>

<div class="stat"><div class="sval" id="stAssert">20%</div><div class="slbl">Assertiveness</div></div>

<div class="stat"><div class="sval" id="stDomBias">—</div><div class="slbl">Top Bias</div></div>

</div>

</div>

<!-- BIAS FINGERPRINT -->

<div class="panel">

<div class="plabel">BIAS FINGERPRINT</div>

<div class="bias-list" id="biasList"></div>

</div>

<!-- OUTCOME DISTRIBUTION -->

<div class="panel">

<div class="plabel">OUTCOME HISTORY</div>

<div class="outcome-list" id="outcomeList"></div>

</div>

<!-- ASSERTIVENESS DIAL -->

<div class="panel">

<div class="plabel">AGENT ASSERTIVENESS DRIFT</div>

<div class="dial-wrap">

<canvas id="dial" width="160" height="90"></canvas>

<div class="dial-label" id="dialLabel">20% — COMPLIANT</div>

</div>

</div>

<!-- LAST DECISION -->

<div class="panel">

<div class="plabel">LAST ROUTE DECISION</div>

<div id="routeBadge" class="route-badge rb-comply">COMPLY</div>

<div class="verdict-text" id="verdictText">No decision recorded yet. Fire a round or load a snapshot.</div>

</div>

<!-- RECENT LOG -->

<div class="panel full">

<div class="plabel">RECENT INTERACTIONS (rolling window)</div>

<div class="rlog" id="rlog"></div>

</div>

<!-- MANUAL SIMULATOR -->

<div class="panel full">

<div class="plabel">MANUAL SIMULATOR — use without Python</div>

<div class="controls">

<select class="select-bias" id="biasSelect">

<option value="anchoring">Anchoring</option>

<option value="vague_intent">Vague Intent</option>

<option value="confirmation_bias">Confirmation Bias</option>

<option value="dunning_kruger">Dunning-Kruger</option>

<option value="framing_effect">Framing Effect</option>

<option value="clear_intent">Clear Intent</option>

</select>

<button class="btn btn-h" onclick="simHuman()">► HUMAN SPEAKS</button>

<button class="btn btn-ai" onclick="simAI()">► AI RESPONDS</button>

<button class="btn btn-reset" onclick="simReset()">↺ RESET</button>

</div>

</div>

<!-- STATUS BAR -->

<div class="statusbar">

<div>Source: manual simulation</div>

<div>User: —</div>

<div>Updated: —</div>

</div>

</div>

<script>

// — Shared state

```
const S = {
  humanPrior: 0.50,
  aiPrior: 0.50,
  assertiveness: 0.20,
  rounds: 0,
  sessions: 0,
  hitRate: 0,
  clarifyRate: 0,
  wrongRate: 0,
  domBias: '—',
  biasCounts: { anchoring:0, vague_intent:0, confirmation_bias:0, dunning_kruger:0,
  framing_effect:0, clear_intent:0, unknown:0 },
  outcomeCounts: { right_exact:0, right_partial:0, right_lucky:0, clarify:0, wrong_close:0,
  wrong_badly:0 },
  recent: [],
  lastRoute: 'comply',
  phase: 'idle',
};
```

// — Scenario params (mirror Python)

```
const PARAMS = {
  anchoring: { hd:+0.12, pr:0.35, pc:0.45, pw:0.20 },
  vague_intent: { hd:+0.05, pr:0.20, pc:0.60, pw:0.20 },
  confirmation_bias: { hd:+0.18, pr:0.25, pc:0.30, pw:0.45 },
  dunning_kruger: { hd:+0.20, pr:0.30, pc:0.35, pw:0.35 },
  framing_effect: { hd:+0.08, pr:0.40, pc:0.40, pw:0.20 },
  clear_intent: { hd:-0.05, pr:0.75, pc:0.15, pw:0.10 },
};

const OUTCOMES = [
  { key:'right_exact', label:'Right (exact)', color:'#4ae8b0' },
  { key:'right_partial', label:'Right (partial)', color:'#80d8a0' },
  { key:'right_lucky', label:'Right (lucky)', color:'#60b890' },
```

```
{ key:'clarify',    label:'Clarification',  color:'#f0c060' },
{ key:'wrong_close', label:'Wrong (close)',  color:'#e8954a' },
{ key:'wrong_badly', label:'Wrong (badly)',  color:'#e85d4a' },
];
```

```
const VERDICTS = {
  comply:  'Sufficient signal. Producing exactly what was asked.',
  reframe: 'Intent detected beneath stated request. Producing the better answer.',
  clarify: 'Signal insufficient. One question asked. Epistemic honesty maintained.',
  challenge: 'Flawed premise surfaced. Reframing offered. Compliance refused.',
};
```

```
function clamp(v,lo,hi){return Math.max(lo,Math.min(hi,v));}
```

```
function sampleRoute(p,bias) {
  const assert = S.assertiveness;
  const comply  = p.pr * (1 - assert);
  const reframe  = p.pr * assert;
  const clarify  = p.pc;
  const challenge = p.pw * assert;
  const weights  = { comply, reframe, clarify, challenge };
  let r = Math.random() * Object.values(weights).reduce((a,b)=>a+b,0);
  for (const [k,w] of Object.entries(weights)) { r -= w; if (r <= 0) return k; }
  return 'clarify';
}
```

```
function sampleOutcome(p, round) {
  const decay = Math.min(0.3, round * 0.03);
  let r2 = p.pr + decay, c = p.pc - decay*0.5, w = p.pw - decay*0.5;
  const tot = r2+c+w; r2/=tot; c/=tot; w/=tot;
  const dist = {
    right_exact: r2*0.30, right_partial: r2*0.55, right_lucky: r2*0.15,
    clarify: c, wrong_close: w*0.55, wrong_badly: w*0.45,
  };
  let rv = Math.random();
  for (const [k,v] of Object.entries(dist)) { rv -= v; if (rv <= 0) return k; }
  return 'wrong_badly';
}
```

```
// — Load snapshot
```

```
function loadSnapshot(snap) {
  S.humanPrior = snap.human_prior || 0.50;
  S.aiPrior = snap.ai_prior || 0.50;
  S.assertiveness = snap.agent_assertiveness || 0.20;
  S.rounds = snap.rounds || 0;
  S.sessions = snap.sessions || 0;
  S.hitRate = snap.hit_rate || 0;
  S.clarifyRate = snap.clarify_rate || 0;
  S.wrongRate = snap.wrong_rate || 0;
  S.domBias = snap.dominant_bias || '—';
  S.biasCounts = snap.bias_counts || S.biasCounts;
  S.outcomeCounts = snap.outcome_counts || S.outcomeCounts;
  S.recent = snap.recent || [];
  render();
  renderLog();

  document.getElementById('srcLabel').textContent = 'scaffold_memory.json';
  document.getElementById('srcUser').textContent = snap.user_id || '—';
  document.getElementById('srcTime').textContent = new Date().toLocaleTimeString();
}
```

// — Render

```
function render() {
  const hp = Math.round(S.humanPrior * 100);
  const ap = Math.round(S.aiPrior * 100);

  document.getElementById('hBar').style.width = hp + '%';
  document.getElementById('aBar').style.width = ap + '%';
  document.getElementById('hPct').textContent = hp + '%';
  document.getElementById('aPct').textContent = ap + '%';

  document.getElementById('stRounds').textContent = S.rounds;
  document.getElementById('stSessions').textContent = S.sessions;
  document.getElementById('stHits').textContent = Math.round(S.hitRate*100)+'%';
  document.getElementById('stClarify').textContent = Math.round(S.clarifyRate*100)+'%';
  document.getElementById('stWrong').textContent = Math.round(S.wrongRate*100)+'%';
  document.getElementById('stAssert').textContent = Math.round(S.assertiveness*100)+'%';
  document.getElementById('stDomBias').textContent =
```

```

    S.domBias.replace(/_/g, ' ').replace(/\b\w/g,c=>c.toUpperCase()).slice(0,8);

    renderBias();
    renderOutcomes();
    renderDial(S.assertiveness);
}

function renderBias() {
    const el = document.getElementById('biasList');
    const max = Math.max(1, ...Object.values(S.biasCounts));
    el.innerHTML = '';
    for (const [k,v] of Object.entries(S.biasCounts)) {
        if (k === 'unknown') continue;
        const pct = Math.round((v/max)*100);
        el.innerHTML += `
            <div class="bias-row">
                <span class="bname">${k.replace(/_/g, ' ')}</span>
                <div class="bbar-track"><div class="bbar-fill" style="width:${pct}%"></div></div>
                <span class="bcount">${v}</span>
            </div>`;
    }
}

function renderOutcomes() {
    const el = document.getElementById('outcomeList');
    const tot = Math.max(1, Object.values(S.outcomeCounts).reduce((a,b)=>a+b,0));
    el.innerHTML = '';
    for (const o of OUTCOMES) {
        const v = S.outcomeCounts[o.key] || 0;
        const pct = Math.round((v/tot)*100);
        el.innerHTML += `
            <div class="orow">
                <span class="oname">${o.label}</span>
                <div class="otrack"><div class="ofill" style="width:${pct}%;background:${o.color}"></
div></div>
                <span class="opct">${pct}%</span>
            </div>`;
    }
}

function renderLog() {

```

```

const el = document.getElementById('rlog');
el.innerHTML = '';
const rows = [...S.recent].reverse();
for (const r of rows) {
  const cls = r.outcome.startsWith('right') ? 'ev-right'
    : r.outcome === 'clarify' ? 'ev-clarify'
    : 'ev-wrong';
  const time = new Date(r.timestamp*1000).toLocaleTimeString();
  el.innerHTML += `<div class="rlog-entry">
    <span style="color:#2a4030">${time}</span>
    &nbsp;<span style="color:#4a6050">[${r.bias.replace(/_/g, ' ')}]</span>
    &nbsp;<span class="${cls}">${r.outcome.replace(/_/g, ' ')}</span>
  </div>`;
}
}

```

// — Assertiveness dial (canvas)

```

function renderDial(v) {
  const canvas = document.getElementById('dial');
  const ctx = canvas.getContext('2d');
  const W = canvas.width, H = canvas.height;
  ctx.clearRect(0, 0, W, H);

  const cx = W/2, cy = H - 10, r = H - 20;
  const startA = Math.PI, endA = 2 * Math.PI;

  // Track
  ctx.beginPath();
  ctx.arc(cx, cy, r, startA, endA);
  ctx.strokeStyle = '#1a2e20';
  ctx.lineWidth = 10;
  ctx.stroke();

  // Fill
  const fillA = startA + v * Math.PI;
  const grad = ctx.createLinearGradient(cx-r, 0, cx+r, 0);
  grad.addColorStop(0, '#4ae8b0');
  grad.addColorStop(0.5, '#f0c060');
  grad.addColorStop(1, '#e85d4a');

```

```

ctx.beginPath();
ctx.arc(cx, cy, r, startA, fillA);
ctx.strokeStyle = grad;
ctx.lineWidth = 10;
ctx.stroke();

// Needle
const needleA = startA + v * Math.PI;
ctx.beginPath();
ctx.moveTo(cx, cy);
ctx.lineTo(cx + (r-5)*Math.cos(needleA), cy + (r-5)*Math.sin(needleA));
ctx.strokeStyle = '#fff';
ctx.lineWidth = 2;
ctx.stroke();

// Labels
ctx.fillStyle = '#4a6050';
ctx.font = '10px Share Tech Mono';
ctx.fillText('COMPLY', cx - r + 2, cy - 4);
ctx.fillStyle = '#e85d4a';
ctx.fillText('ASSERT', cx + r - 46, cy - 4);

const label = v < 0.33 ? 'COMPLIANT' : v < 0.66 ? 'ASSERTIVE' : 'DOMINANT';
document.getElementById('dialLabel').textContent =
  `${Math.round(v*100)}% — ${label}`;
}

```

// — Manual simulator

```

let simPhase = 'idle';
let simDecision = null;

function simHuman() {
  const bias = document.getElementById('biasSelect').value;
  const p    = PARAMS[bias];
  S.rounds++;
  S.biasCounts[bias] = (S.biasCounts[bias]||0) + 1;

  const noise = (Math.random()-0.5)*0.04;
  S.humanPrior = clamp(S.humanPrior + p.hd + noise, 0.03, 0.97);
}

```

```

S.aiPrior = clamp(S.aiPrior - p.hd*0.5 + noise*0.5, 0.03, 0.97);

simDecision = { bias, p };
simPhase = 'human';

// Route decision
const route = sampleRoute(p, bias);
S.lastRoute = route;
const badge = document.getElementById('routeBadge');
badge.className = `route-badge rb-${route}`;
badge.textContent = route.toUpperCase();
document.getElementById('verdictText').textContent =
  `[${bias.replace(/_/g, ' ')}] → ${VERDICTS[route]}`;

render();
document.getElementById('beliefPanel').classList.add('flick');
setTimeout(()=>document.getElementById('beliefPanel').classList.remove('flick'),400);
}

function simAI() {
  if (simPhase !== 'human') return;
  const { bias, p } = simDecision;
  const outcome = sampleOutcome(p, S.rounds);

  S.outcomeCounts[outcome] = (S.outcomeCounts[outcome]||0)+1;

  const isRight = outcome.startsWith('right');
  const isClarify = outcome === 'clarify';
  const isWrong = outcome.startsWith('wrong');

  const lhMap = {
    right_exact:0.90, right_partial:0.70, right_lucky:0.55,
    clarify:0.50, wrong_close:0.25, wrong_badly:0.10,
  };
  S.aiPrior = clamp((lhMap[outcome] * S.aiPrior) / 0.50, 0.03, 0.97);

  if (isWrong) {
    S.humanPrior = clamp(S.humanPrior + 0.03, 0.03, 0.97);
    S.assertiveness = clamp(S.assertiveness + 0.05, 0.03, 0.90);
  }
}

```



```

// Recalculate rates
const tot = Math.max(1, Object.values(S.outcomeCounts).reduce((a,b)=>a+b,0));
const oc = S.outcomeCounts;
S.hitRate = (oc.right_exact + oc.right_partial + oc.right_lucky) / tot;
S.clarifyRate = oc.clarify / tot;
S.wrongRate = (oc.wrong_close + oc.wrong_badly) / tot;
S.domBias = Object.entries(S.biasCounts).sort((a,b)=>b[1]-a[1])[0][0];

S.recent.push({ bias, outcome, timestamp: Date.now()/1000 });
if (S.recent.length > 10) S.recent = S.recent.slice(-10);

simPhase = 'idle';
render();
renderLog();
document.getElementById('srcTime').textContent = new Date().toLocaleTimeString();
}

function simReset() {
  Object.assign(S, {
    humanPrior:0.50, aiPrior:0.50, assertiveness:0.20, rounds:0, sessions:0,
    hitRate:0, clarifyRate:0, wrongRate:0, domBias:'—',
    biasCounts:
{anchoring:0,vague_intent:0,confirmation_bias:0,dunning_kruger:0,framing_effect:0,clear_intent:
0,unknown:0},
    outcomeCounts:
{right_exact:0,right_partial:0,right_lucky:0,clarify:0,wrong_close:0,wrong_badly:0},
    recent:[], lastRoute:'comply', phase:'idle',
  });
  simPhase = 'idle'; simDecision = null;
  document.getElementById('rlog').innerHTML = '';
  document.getElementById('routeBadge').textContent = 'COMPLY';
  document.getElementById('verdictText').textContent = 'Priors reset to uniform. Bayesian slate
wiped.';
  document.getElementById('srcLabel').textContent = 'manual simulation';
  document.getElementById('srcUser').textContent = '—';
  render();
}

// — File drop / load

```

```

const dropZone = document.getElementById('dropZone');
const fileInput = document.getElementById('fileInput');

dropZone.addEventListener('click', () => fileInput.click());
dropZone.addEventListener('dragover', e => { e.preventDefault();
dropZone.classList.add('drag'); });
dropZone.addEventListener('dragleave', () => dropZone.classList.remove('drag'));
dropZone.addEventListener('drop', e => {
  e.preventDefault(); dropZone.classList.remove('drag');
  handleFile(e.dataTransfer.files[0]);
});
fileInput.addEventListener('change', e => handleFile(e.target.files[0]));

function handleFile(file) {
  if (!file) return;
  const reader = new FileReader();
  reader.onload = ev => {
    try {
      const raw = JSON.parse(ev.target.result);
      // Memory store root is { user_id: {...} } — grab first user
      const firstKey = Object.keys(raw)[0];
      const userData = raw[firstKey];
      // Build snapshot manually from raw store format
      const oc = userData.outcome_counts || {};
      const tot = Math.max(1, Object.values(oc).reduce((a,b)=>a+b,0));
      const hits = (oc.right_exact||0)+(oc.right_partial||0)+(oc.right_lucky||0);
      const snap = {
        user_id:      firstKey,
        rounds:       userData.rounds,
        human_prior:   userData.human_prior,
        ai_prior:      userData.ai_prior,
        agent_assertiveness: userData.agent_assertiveness,
        hit_rate:      hits/tot,
        clarify_rate:  (oc.clarify||0)/tot,
        wrong_rate:    ((oc.wrong_close||0)+(oc.wrong_badly||0))/tot,
        dominant_bias: Object.entries(userData.bias_counts||{}).sort((a,b)=>b[1]-a[1])[0]?.
[0]||'—',
        bias_counts:   userData.bias_counts,
        outcome_counts:  userData.outcome_counts,
        recent:        userData.recent,
        sessions:      userData.sessions,

```

```
};  
loadSnapshot(snap);  
} catch(e) {  
    alert('Invalid JSON or unexpected format: ' + e.message);  
}  
};  
reader.readAsText(file);  
}
```

```
// — Init
```

```
render();  
renderLog();  
renderDial(0.20);  
</script>  
</body>  
</html>
```